KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur, Virudhunagar (Dt.) - 626126 | OMR, Chennai, Tamil Nadu
info@kalasalingam.ac.in | www.kalasalingam.ac.in

School ofComputing
Department of ComputerScience & Engineering

CAMPUS EVENT PORTAL

Mini ProjectReport

Submitted by
9925051033 - A.Logeswari
9925051090 - B.Geetha Hasini
9925051001 - J.P. Adlin Breesha
9925051017 - Ch. Venkata sai rajesh
9925051092 - K.Jaswanth

In partial fulfilment for the award of the degree of

Bachelor Of Technology

In

Computer Science and Engineering

SCHOOL OF COMPUTING COMPUTER SCIENCE

AND ENGINEERING

KALASALINGAMACADEMY OF RESEARCH AND EDUCATION

KRISHNANKOIL 626 126

November 2026

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur, Virudhunagar (Dt.) - 626126 | OMR, Chennai, Tamil Nadu
info@kalasalingam.ac.in | www.kalasalingam.ac.in

DECLARATION

We hereby declare that this mini project **"CAMPUS EVENT PORTAL"** is our genuine work carried out by us under the guidance of **Mrs.S.Reshni, Assistant Professor**, Department of Computer Science and Engineering, **Kalasalingam Academy of Research and Education.**

We declare that this project has been developed as part of the academic requirements for the **Object Oriented Programming using Java (CSE203)** course. The work presented in this report is our own, and all sources of information, references, and resources used in the development of the project have been duly acknowledged. We further declare that we have contributed to the design, development, implementation, testing, and documentation of the project according to our assigned roles. We take responsibility for the authenticity of the work submitted and confirm that this project has not been submitted previously, either in whole or in part, for any other academic assessment.

REGISTER NUMBER	*STUDENT NAME*	*STUDENT SIGNATURE*
9925051033	LOGESWARI A	
9925051090	GEETHA HASINIB	
9925051001	ADLIN BREESHAJP	
9925051017	VENKATA SAI RAJESHCH	
9925051092	JASWANTH K	

Date of End Semester Project Review:

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur, Virudhunagar (Dt.) - 626126 | OMR, Chennai, Tamil Nadu
info@kalasalingam.ac.in | www.kalasalingam.ac.in

BONAFIDE CERTIFICATE

Certified that this report titled Campus Event Portal **,** for the Project based Learning, is a *Bonafide* work of A.LOGESWARI **(9925051033), B.GEETHA HASINI (9925051090), J.P.ADLIN BREESHA(9925051001)** , C.H. VENKATA SAI RAJESH (99250511017), K.JASWANTH (9925051092) who carried out the work together as a team together for the fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Data Science , during the Academic year Odd Semester (2026-2027).

Place: Krishnankoil

Date:

Mrs.S.Reshni, ME.,
AssistantProfessor,Department of Computer Science & Engineering, Kalasalingam Academy of Research & Education, Krishnan Koil - 626126.

Dr. R. Raja Subramanian, M.E., Ph. D.,
Professor and Head,DepartmentofComputerScience & Engineering, Kalasalingam Academy of Research & Education, Krishnan Koil - 626126.

ACKNOWLEDGEMENT

We express our sincere gratitude to ourinstitutionand theDepartment ofComputer Science and Engineering for providing us with the opportunity and resources to undertake this mini project as part of the Java Programming course.

We extend our heartfelt thanks to our Head of the Department, **Dr. R. Raja Subramanian, M.E., Ph. D.,** for the encouragement and support provided throughout the project. We are grateful to our course faculty and project guide **Mrs.S.Reshni, Assistant Professor,** for her valuable guidance, constructive suggestions, and continuous support during the design, development, implementation, and testing of the project.

We also thank our classmates and team members for their cooperation, teamwork, and contributions toward the successful completion of this project.

Finally, we express our gratitude to everyone who directly or indirectly supported us in completing this mini project successfully.

TABLE OF CONTENTS

CHAPTER NO	CONTENTS	PAGE NO
1.	**Introduction**	**7**
	1.1 Problem Statement	7
	1.2 Objectives of the Project	7
	1.3 Scope of the Project	8
	1.4 Overview of the Event Management System	8
2.	**Existing & Proposed System**	**9**
	2.1 Existing System	9
	2.2 Limitations of the Existing System	9
	2.3 Proposed System	9
	2.4 Advantages of the Proposed System	10
	2.5 Comparison of Existing and Proposed System	10
3.	**Requirements**	**11**
	3.1 Hardware Requirements	11
	3.2 Software Requirements	11
	3.3 Functional Requirements	11
	3.4 Non-Functional Requirements	12
	3.5 Technologies Used	12
4.	**System Architecture & Design**	**13**
	4.1 System Architecture	13
	4.2 System Modules	13
	4.3 Application Workflow	14
	4.4 System Flowchart	15
	4.5 Database Design Overview	16
5.	**OOP Concepts Used**	**17**
	5.1 Classes and Objects	17
	5.2 Encapsulation	17
	5.3 Abstraction	17
	5.4 Inheritance	17
	5.5 Polymorphism	17
	5.6 Application of OOP Concepts in the Project	18
6.	**Exception Handling & User-Defined Exceptions**	**19**
	6.1 Exception Handling in Java	19
	6.2 Exception Handling in the Project	19
	6.3 Built-in Exceptions	19
	6.4 User-Defined Exceptions	19
	6.5 Error Handling and Validation	20

TABLE OF CONTENTS

CHAPTER NO	CONTENTS	PAGE NO
7.	**Collections & Data Structures / AI**	**21**
	7.1 Java Collections Framework	21
	7.2 Data Structures Used in the Project	21
	7.3 Data Management Using Java Objects	21
	7.4 AI / Machine Learning Applicability	21
8.	**JDBC & Database**	**23**
	8.1 Introduction to JDBC	23
	8.2 Database Connectivity	23
	8.3 Database Schema and Tables	23
	8.4 Database Operations (CRUD)	24
	8.5 Prepared Statements	24
	8.6 Database Initialization and Data Management	24
9.	**Graphical User Interface**	**25**
	9.1 User Interface Design	25
	9.2 Home Page and Dashboard	25
	9.3 Event Management Interface	25
	9.4 Registration Management Interface	25
	9.5 Budget and Feedback Interface	25
	9.6 Leaderboard and Recommendation Interface	25
	9.7 Event Statistics and Overview	26
10.	**GitHub Details & Implementation**	**27**
	10.1 GitHub Repository Details	27
	10.2 Project Structure and Package Organization	27
	10.3 Important Classes and Their Responsibilities	27
	10.4 Implementation of Major Modules	28
	10.5 Database Configuration and Setup	28
	10.6 Steps to Run the Application	28
11.	**Testing**	**30**
	11.1 Testing Methodology	30
	11.2 Test Cases	30
	11.3 Functional Testing	30
	11.4 Database Testing	30
	11.5 Integration Testing	31
	11.6 Test Results	31
12.	**Results & Screenshots**	**32**
	12.1 Project Results	32
	12.2 Home Page and Dashboard	34
	12.3 Event Management Screenshots	35
	12.4 Registration Management Screenshots	36
	12.5 Budget and Feedback Screenshots	37
	12.6 Leaderboard and Recommendation Screenshots	39
13.	**Conclusion**	**41**
	13.1 Project Summary	41
	13.2 Learning Outcomes	41
14.	**Future Enhancement**	**42**
	14.1 AI-Based Personalized Event Recommendations	42
	14.2 Automated Email and SMS Notifications	42
	14.3 Smart Student Preference and Personalized Event Assistant	42

1. INTRODUCTION

Colleges conduct a large number of technical, cultural and training events every semester. Registration, seat control, attendance, budgets and feedback are normally handled through forms, messages and spreadsheets, which is slow and error-prone. The Campus Event Management System is a Java application that manages the complete life cycle of a college event on one platform: event creation, student registration, waiting list, attendance, rewards, budget approval and feedback.

The project is inspired by the swipin event portal and removes six drawbacks that were observed in it: timetable clashes, unused cancelled seats, low student motivation, difficulty in finding suitable events, spreadsheet-based budgeting and feedback that does not explain satisfaction. The system is built with Java, JDBC and MySQL and follows the layered flow JavaFX/Browser → Controller → Model → Service → DAO → Database.

1.1 Problem Statement

Existing event portals only list events and accept registrations. They do not warn a student when two events overlap, they leave seats empty when a registered student cancels, they give no reward for participation, they show every event to every student, they keep budgets outside the system and they collect feedback that is not anonymous or detailed. As a result organizers lose time, students miss useful events and the administration has no reliable record of event spending or quality. A single integrated system that solves these problems is required.

1.2 Objectives of the Project

1. To develop a Java application for managing college events with a MySQL database through JDBC.
2. To provide role-based access for Students, Organizers and Administrators.
3. To detect timetable clashes and warn the student before registration.
4. To maintain an automatic waiting list and notify the next eligible student when a seat becomes free.
5. To award participation points and badges and display a department-wise leaderboard.
6. To recommend events using department, interests, year of study and participation history.
7. To manage budget requests, expense records and bill uploads with administrator approval.
8. To collect anonymous feedback, ratings and suggestions and generate event-quality reports.
9. To apply object-oriented concepts such as abstraction, inheritance, polymorphism and encapsulation.

10. To follow a clean layered architecture that separates user interface, business logic and data access.

1.3 Scope of the Project

The system covers the following areas:

- **User management:** registration, login and role-based dashboards.
- **Event management:** creation, listing, search, filtering and eligibility rules by department and year.
- **Registration management:** registration, cancellation, clash detection, seat control and waiting list.
- **Gamification:** points, badges, department leaderboard and top students.
- **Recommendation:** personalised event suggestions with the reason shown.
- **Budget management:** budget requests, expenses, bill upload and approval.
- **Feedback and reports:** anonymous feedback and text-based event-quality reports.
- **Notifications:** in-app messages for seat promotion, approvals and badges.

The project is intended for use inside a single college. Online payment, e-mail/SMS delivery and mobile applications are outside the present scope.

1.4 Overview of the Event Management System

The application has two front-ends that share one Java back-end: a browser-based website served by the JDK built-in HTTP server and an optional JavaFX desktop application. In both cases the front-end only collects input and displays results. The Controller receives the request, the Model stores the data using setters, the Service validates and processes it, and the DAO reads the Model with getters and stores it in MySQL using JDBC. The same back-end therefore works for either front-end without any change.

2. EXISTING & PROPOSED SYSTEM

2.1 Existing System

In the existing portal students browse a list of events and register for them. Organizers receive the registrations and handle attendance, budgets and feedback outside the portal using paper forms, messaging groups and spreadsheets. The system works for small events but does not support the decision-making and control that a college with many parallel events needs.

2.2 Limitations of the Existing System

1. **Timetable conflicts:** a student can register for two events at the same time and discovers the clash only on the event day.
2. **Wasted seats:** when a registered student cancels, the seat stays empty because no waiting list exists.
3. **Low motivation:** there is no recognition for students who participate regularly.
4. **Poor discovery:** every student sees the same list, so relevant events are easily missed.
5. **Manual budgeting:** budget requests and bills are kept in spreadsheets and papers, which are hard to track and verify.
6. **Weak feedback:** a simple star rating does not explain why students were satisfied or dissatisfied.
7. **No reports:** organizers have no summary of event quality or student response.
8. **Scattered data:** information is stored in many places, which causes duplication and loss of records.

2.3 Proposed System

1. **Clash detection:** the system compares date and time of the selected event with the student's registered events and blocks the registration with a warning.
2. **Automatic waiting list:** a full event adds the student to a waiting list; on cancellation the first eligible student gets the seat and a notification.
3. **Points, badges and leaderboard:** attended events give points by event type and unlock badges; a department-wise leaderboard is displayed.
4. **Recommendations:** events are scored using department, interests, year and history and shown with the reason.
5. **Budget workflow:** organizers submit budget requests and expenses with bills; the administrator approves or rejects them and a budget-versus-spending summary is shown.
6. **Anonymous feedback:** only attendees can give rating, comment and suggestion once, without their identity being stored in the feedback record.

7. **Event-quality report:** average rating, response count and a quality grade are generated for every event.

2.4 Advantages of the Proposed System

1. Prevents students from joining overlapping events.
2. Uses every seat through the automatic waiting list.
3. Encourages participation through points and badges.
4. Saves time by suggesting relevant events.
5. Keeps all budget and bill records in one place.
6. Gives honest feedback because identity is not stored.
7. Gives organizers a clear quality report for improvement.
8. Uses a layered design that is easy to maintain and extend.
9. Works in a browser and as a desktop application on the same back-end.

2.5 Comparison of Existing and Proposed System

S.NO	Existing System	Proposed System
1	Timetable clash is not checked	Clash is detected and registration is blocked with a warning
2	Cancelled seats remain empty	Automatic waiting list promotes the next eligible student
3	No reward for participation	Points, badges and department leaderboard
4	Same event list for all students	Recommendations by department, interests, year and history
5	Budget kept in spreadsheets	Budget request, expense, bill upload and admin approval
6	Simple rating only	Anonymous rating, comment and suggestion
7	No event report	Event-quality report with average rating and grade
8	Data stored in many places	Single MySQL database accessed through JDBC
9	No defined software structure	Layered flow: Controller, Model, Service, DAO

3. REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB (8 GB recommended)
Hard Disk	500 MB free space (plus space for uploaded bills)
Display	1280 x 720 resolution
Network	Local machine / college LAN

3.2 Software Requirements

Software	Description
Operating System	Windows 10/11, Linux or macOS
Java	JDK 17 or above (JDK 21 recommended)
Database	MySQL Server 8.x
JDBC Driver	MySQL Connector/J
GUI (optional)	JavaFX SDK for the desktop version
Browser	Chrome, Edge or Firefox for the website
Editor / IDE	VS Code, IntelliJ IDEA or Eclipse

3.3 Functional Requirements

ID	Requirement	Description
FR1	User registration and login	Users create an account and log in with email and password.
FR2	Event management	Organizers create events with type, date, time, venue, capacity and eligibility.
FR3	Registration	Students register and cancel registrations for eligible events.
FR4	Clash detection	System warns and blocks registration of overlapping events.
FR5	Waiting list	Full events add students to a waiting list and promote them on cancellation.
FR6	Attendance and points	Organizers mark attendance; points and badges are awarded.
FR7	Leaderboard	Department-wise ranking and top students are displayed.
FR8	Recommendation	Events are suggested using a scoring rule.

ID	Requirement	Description
FR9	Budget	Budget requests and expenses with bill upload; admin approval.
FR10	Feedback	Anonymous rating, comment and suggestion; event-quality report.
FR11	Notifications	Users view messages about seats, approvals and badges.

3.4 Non-Functional Requirements

1. **Usability:** simple screens with clear messages and warnings.
2. **Performance:** pages and operations respond within a second for college-level data.
3. **Security:** passwords are stored as hashes; role checks protect organizer and admin features.
4. **Reliability:** multi-table updates such as feedback use transactions.
5. **Maintainability:** layered packages allow each part to change independently.
6. **Portability:** runs on any operating system with Java and MySQL.
7. **Data integrity:** primary keys, foreign keys and validation prevent invalid records.
8. **Privacy:** feedback records contain no student identity.

3.5 Technologies Used

Technology	Purpose
Java	Core language, OOP, collections, exception handling
JDBC	Connection between Java and MySQL
MySQL	Relational database (event_ms)
JDK HttpServer	Serves the website and JSON requests
HTML, CSS, JavaScript	Browser front-end
JavaFX	Desktop front-end (optional)

4. SYSTEM ARCHITECTURE & DESIGN

4.1 System Architecture

The system follows a layered architecture. The front-end collects the input, the Controller receives it, the Model stores it using setters, the Service validates and processes it, and the DAO uses getters and JDBC to store it in the database. Results travel back along the same path. The WebServer class only converts HTTP/JSON messages into Controller calls and contains no business rules or SQL.

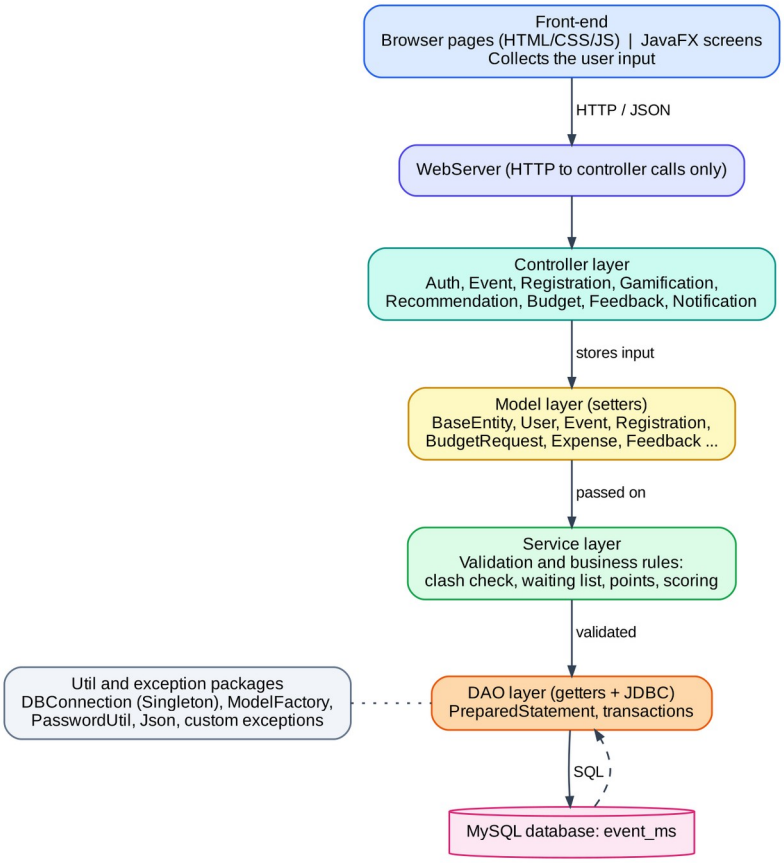


Figure 4.1: System Architecture

4.2 System Modules

1. **Authentication Module:** registration, login and role detection (AuthController, AuthService, UserDAO).
2. **Event Module:** create, list, search and filter events (EventController, EventService, EventDAO).
3. **Registration Module:** registration, cancellation, clash check and waiting list (RegistrationService, RegistrationDAO).
4. **Gamification Module:** points, badges and leaderboard (GamificationService, BadgeDAO).

5. **Recommendation Module:** rule-based scoring of events (RecommendationService).
6. **Budget Module:** budget requests, expenses, bills and approval (BudgetService, BudgetDAO).
7. **Feedback Module:** anonymous feedback and reports (FeedbackService, FeedbackDAO).
8. **Notification Module:** in-app messages (NotificationService, NotificationDAO).

4.3 Application Workflow

The workflow below shows how a user moves from login to the role-based dashboard and then to one of the five functional areas. Every step names the class that handles it.

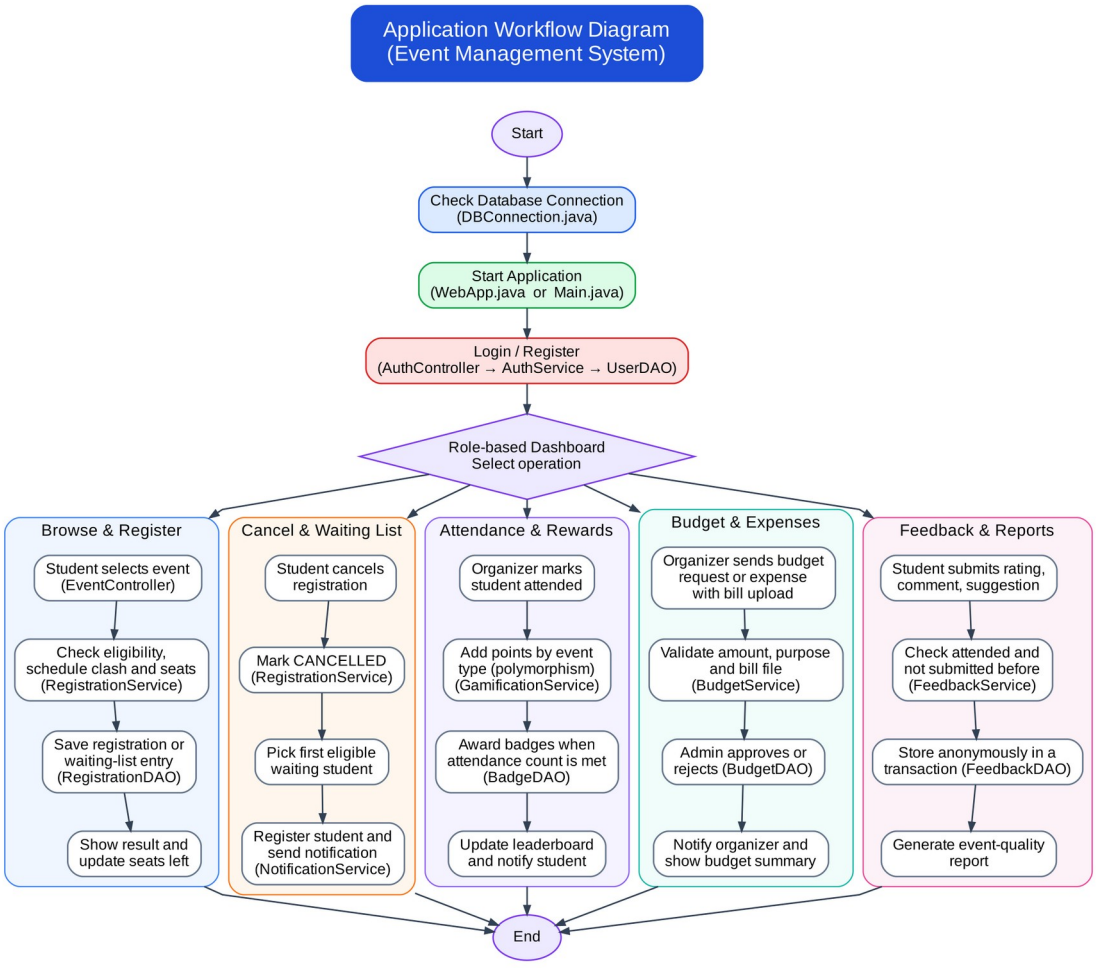


Figure 4.2: Application Workflow Diagram

4.4 System Flowchart

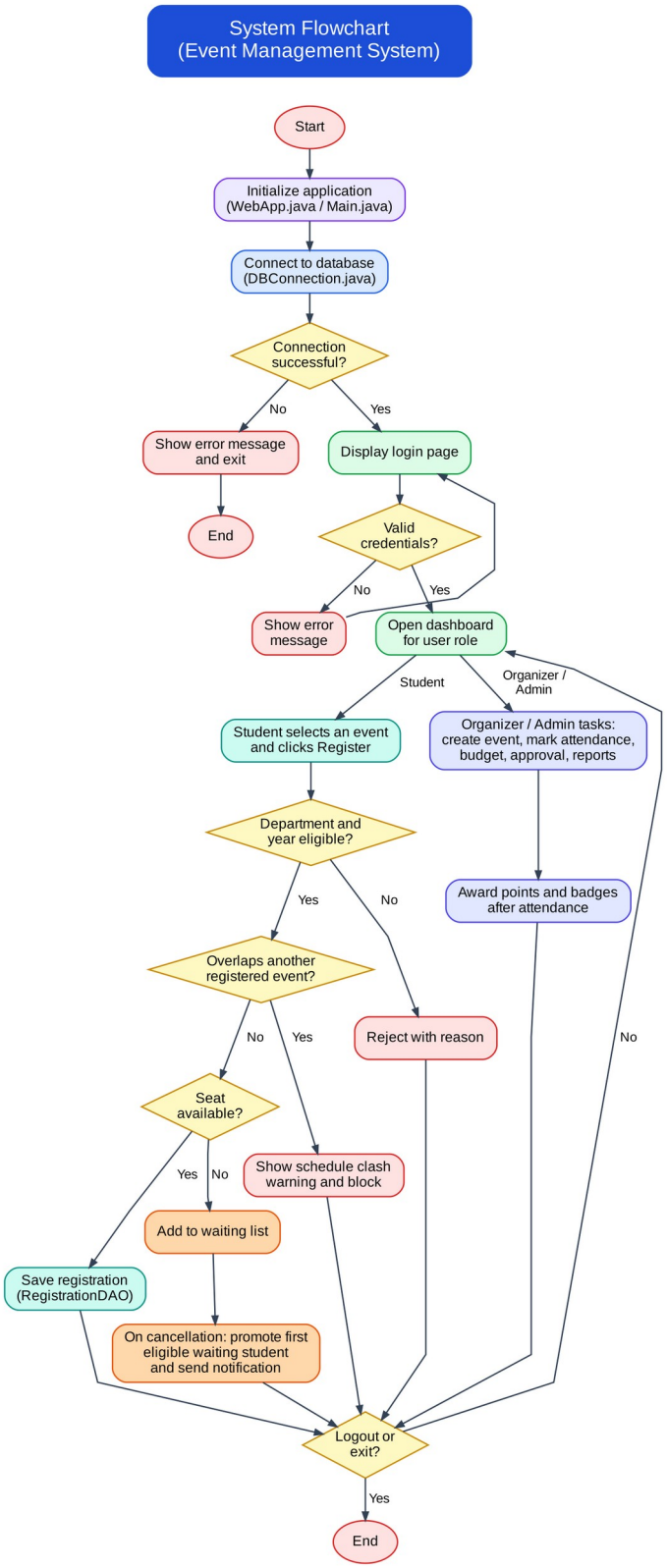


Figure 4.3: System Flowchart

4.5 Database Design Overview

The database event_ms contains ten tables: users, events, registrations, waiting_list, user_badges, notifications, budget_requests, expenses, feedback and feedback_given. Users and events are the main tables; the other tables refer to them through foreign keys. The feedback table intentionally has no student column, and feedback_given records only that a student has submitted feedback to prevent duplicates.

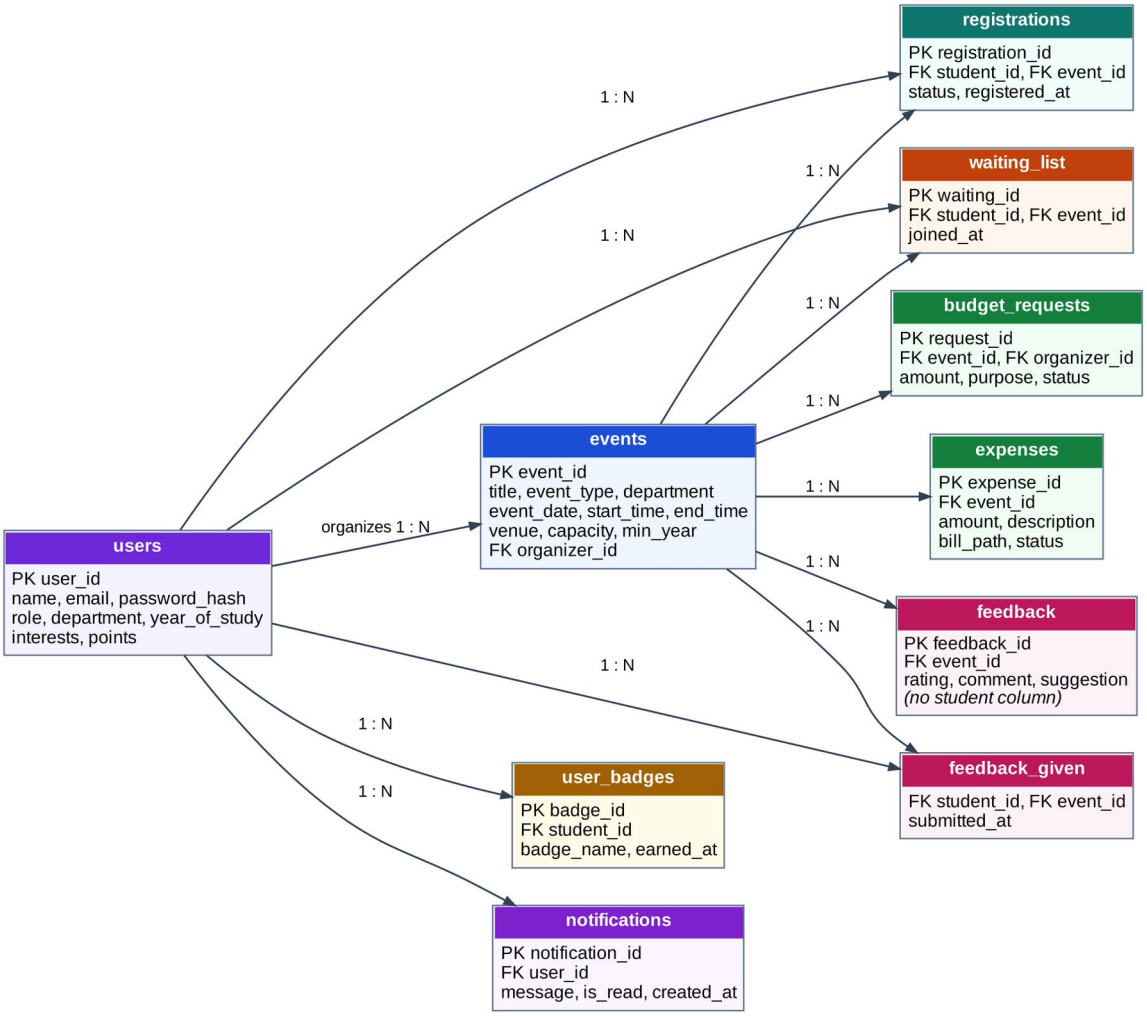


Figure 4.4: Entity Relationship Diagram

5. OOP CONCEPTS USED

5.1 Classes and Objects

A class is a blueprint and an object is an instance of it. The project has model classes such as Student, Organizer, Admin, Workshop, Seminar, Competition, Registration, BudgetRequest, Expense and Feedback. Service classes such as RegistrationService and DAO classes such as EventDAO are also objects created at run time and used by the controllers.

5.2 Encapsulation

Every model class keeps its fields private and exposes them through public getters and setters. The Controller stores input through setters and the DAO reads the data through getters, which hides the internal representation and allows validation inside setters and services.

5.3 Abstraction

BaseEntity, User and Event are abstract classes. They declare abstract methods such as describe(), getRole() and getPointsAwarded() without implementation, and subclasses supply the details. The Validatable interface declares validate() for Event, BudgetRequest, Expense and Feedback.

5.4 Inheritance

Student, Organizer and Admin extend User; Workshop, Seminar and Competition extend Event; and all models extend BaseEntity. Common fields and methods are written once in the parent class and reused by the children.

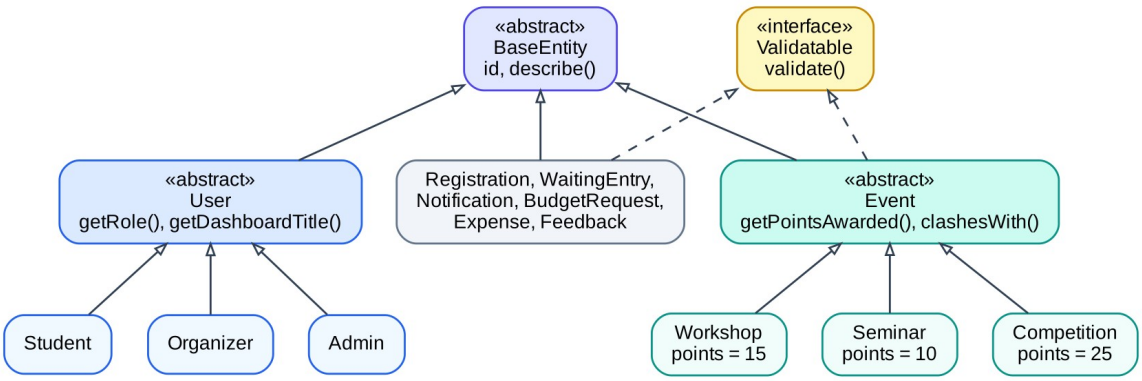


Figure 5.1: Class Hierarchy

5.5 Polymorphism

The same method call behaves differently for different objects. event.getPointsAwarded() returns 15 for a Workshop, 10 for a Seminar and 25 for a Competition, so GamificationService awards points without checking the event type. Likewise user.getRole() and user.getDashboardTitle() decide which dashboard is displayed.

5.6 Application of OOP Concepts in the Project

OOP Concept	Where Used
Abstraction	BaseEntity, User, Event, Validatable
Inheritance	Student/Organizer/Admin extends User; Workshop/Seminar/Competition extends Event
Polymorphism	getPointsAwarded(), getRole(), getDashboardTitle()
Encapsulation	Private fields with getters and setters in all models
Interface	Validatable implemented by four model classes
Singleton pattern	DBConnection.getInstance()
Factory pattern	ModelFactory.createEvent() and createUser()
Custom exceptions	ValidationException and ScheduleClashException extend EventMsException
Enums	Role and Status
Collections	HashMap, HashSet, LinkedHashMap, TreeMap, List with Comparator

6. EXCEPTION HANDLING & USER-DEFINED EXCEPTIONS

6.1 Exception Handling in Java

An exception is an event that interrupts the normal flow of a program. Java handles it with five keywords: try encloses code that may fail, catch handles a specific exception, finally runs cleanup code, throw raises an exception and throws declares that a method may raise one.

```
try {
    Connection con = DBConnection.getInstance().getConnection();
    // database work
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
} finally {
    System.out.println("Operation finished");
}
```

6.2 Exception Handling in the Project

The Service layer throws exceptions when a rule is broken, the DAO layer converts SQL problems into project exceptions and the Controller/WebServer converts any exception into a clear message for the user. This keeps error handling out of the user-interface code.

6.3 Built-in Exceptions

Exception	Where / Why
SQLException	DAO classes during JDBC operations
IOException	Reading db.properties and saving uploaded bills
NumberFormatException	Converting numeric input such as amount or capacity
DateTimeParseException	Parsing event date and time
NullPointerException	Avoided by null checks on optional fields
ClassNotFoundException	Loading the JDBC driver

6.4 User-Defined Exceptions

EventMsException is the base class. ValidationException is thrown for invalid input and ScheduleClashException is thrown when a registration overlaps with another event.

```
public class EventMsException extends RuntimeException {
    public EventMsException(String message) { super(message); }
}
public class ScheduleClashException extends EventMsException {
    public ScheduleClashException(String message) { super(message); }
}
```

6.5 Error Handling and Validation

- Empty fields, invalid email and short passwords are rejected by AuthService.
- Event end time before start time, zero capacity or missing title are rejected by EventService.
- Registration is rejected if department or year is not eligible, if a clash exists or if the student is already registered.
- Feedback is rejected unless the student attended and has not submitted before.
- Budget and expense amounts must be positive and a bill file is validated before saving.

7. COLLECTIONS & DATA STRUCTURES / AI

7.1 Java Collections Framework

The Java Collections Framework provides ready-made data structures such as List, Set and Map. The project uses them to hold database results and to process data in memory before it is shown to the user or saved.

7.2 Data Structures Used in the Project

Collection	Use in the Project
List / ArrayList	Lists of events, registrations, notifications and report rows
HashMap	Mapping an event id to its details and counting registrations
LinkedHashMap	Leaderboard rows that must keep insertion (rank) order
TreeMap	Department-wise grouping in sorted order
HashSet	Set of event ids a student has registered for, for fast lookup
Comparator	Sorting recommendations by score and leaderboard by points

7.3 Data Management Using Java Objects

Data is managed as objects instead of raw table rows. The DAO reads each row of a ResultSet and creates a model object (for example a Workshop, Seminar or Competition through ModelFactory), and the objects are collected in lists and passed to the Service layer. The Service layer works only with these objects, so business rules do not depend on SQL.

- **Clash detection:** two events clash when they are on the same date and the start time of one is before the end time of the other and vice versa (Event.clashesWith).
- **Waiting-list promotion:** waiting entries are read in order of joining and the first student who is eligible and has no clash receives the seat.
- **Badge award:** badges are granted when the attended-event count or points reach the defined limits.
- **Recommendation scoring:** for each upcoming event, points are added for matching department, interests, year of study and previous participation; already registered or clashing events are skipped and the list is sorted by score using a Comparator, with the reason shown.
- **Quality grade:** the average of all ratings is converted into a grade label in the event report.

7.4 AI / Machine Learning Applicability

The current version does not use machine learning. Recommendations are produced by a transparent rule-based scoring method, which is suitable for the small amount of college

data and lets the student see why an event was suggested. Machine learning could be applied in future for better recommendations from participation history, for predicting event attendance and for analysing the sentiment of feedback comments (see Chapter 14).

8. JDBC & DATABASE

8.1 Introduction to JDBC

JDBC (Java Database Connectivity) is the standard Java API for working with relational databases. The project uses the following JDBC classes and interfaces.

Class / Interface	Purpose
DriverManager	Creates a connection to the database
Connection	Represents the session with MySQL; used for transactions
PreparedStatement	Executes parameterised SQL
ResultSet	Holds rows returned by a query
SQLException	Reports database errors

8.2 Database Connectivity

DBConnection is a Singleton class. It reads the URL, user name and password from resources/db.properties and gives the DAO classes a connection. Because only one object is created, the settings are loaded once.

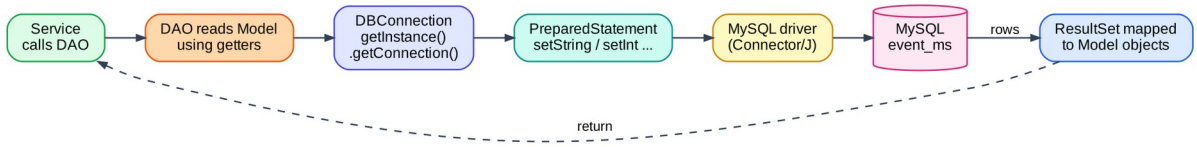


Figure 8.1: Database Connectivity Flow

8.3 Database Schema and Tables

Table	Main Columns	Purpose
users	user_id, name, email, password_hash, role, department, year_of_study, interests, points	Students, organizers and admins
events	event_id, title, event_type, department, event_date, start_time, end_time, venue, capacity, organizer_id	Event details
registrations	registration_id, student_id, event_id, status, attended	Event registrations
waiting_list	waiting_id, student_id, event_id, joined_at	Waiting students
user_badges	badge_id, student_id, badge_name	Earned badges
notifications	notification_id, user_id, message, is_read	Messages
budget_requests	request_id, event_id, amount, purpose, status	Budget approval
expenses	expense_id, event_id, amount, description, bill_path, status	Expense records

Table	Main Columns	Purpose
feedback	feedback_id, event_id, rating, comment, suggestion	Anonymous feedback
feedback_given	student_id, event_id	Duplicate prevention only

8.4 Database Operations (CRUD)

Operation	SQL	Example in the Project
Create	INSERT	Register for an event, create an event, add a budget request
Read	SELECT	List events, leaderboard, notifications
Update	UPDATE	Mark attendance, approve budget, add points
Delete	DELETE	Remove a waiting-list entry after promotion

8.5 Prepared Statements

All queries use PreparedStatement with placeholders. This prevents SQL injection and makes the code easier to read. try-with-resources closes the connection and statement automatically.

```
String sql = "INSERT INTO registrations(student_id, event_id, status)
VALUES (?, ?, ?)";
try (Connection con = DBConnection.getInstance().getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, reg.getStudentId());
    ps.setInt(2, reg.getEventId());
    ps.setString(3, reg.getStatus().name());
    ps.executeUpdate();
}
```

8.6 Database Initialization and Data Management

The file sql/schema.sql creates the event_ms database, all tables and sample data (five users and five events). It is run once with the command: mysql -u root -p < sql/schema.sql. The feedback submission uses a transaction so that the anonymous feedback row and the feedback_given row are saved together or not at all.

9. GRAPHICAL USER INTERFACE

9.1 User Interface Design

The project provides a website that runs in any browser and an optional JavaFX desktop application. Both use the same Controller layer. The interface uses a clean light-purple theme with cards, tabs, coloured status chips, warning banners for schedule clashes, toast messages for success and error, and a star-rating dialog. The layout adapts to small screens. The website is built with HTML, CSS and JavaScript (index.html, style.css, app.js) and the desktop version with JavaFX (Main, LoginView, StudentDashboard, StaffDashboard).

9.2 Home Page and Dashboard

A split login screen shows the project highlights on the left and a login / sign-up card on the right. Sign-up asks for name, email, password, role, department and year. After login each role gets its own dashboard and tab bar: students see their points, registrations and notification count, organizers see their events and tools, and administrators see overall counts.

9.3 Event Management Interface

Students use the Events tab with search, type and department filters; every event card shows date, time, venue, seats left and a clash warning where needed. Organizers use Create Event, My Events and Attendance tabs to create events and mark students as attended.

9.4 Registration Management Interface

Students register or join the waiting list from the event card, cancel from My Events, view the Waiting List tab and receive notifications such as "A seat opened up!" when promoted.

9.5 Budget and Feedback Interface

Organizers submit budget requests and expenses with a bill file and see approval status and balance. Administrators approve or reject from the Budget Approvals tab. Students give anonymous feedback from My Events, and organizers and administrators open the Reports tab for the event-quality report.

9.6 Leaderboard and Recommendation Interface

The Recommended tab lists suggested events with the reason. The Profile tab shows points and earned badges, and the Leaderboard tab shows the department-wise ranking and top students.

9.7 Event Statistics and Overview

The administrator dashboard shows totals for events, students, registrations and pending budget requests, and the budget summary shows approved budget, expenses and balance for each event.

10. GITHUB DETAILS & IMPLEMENTATION

10.1 GitHub Repository Details

Item	Details
Project Name	Campus Event Management System
Repository URL	https://github.com/<your-username>/ EventManagementSystem (to be filled)
Language	Java 21, SQL, HTML/CSS/JavaScript
Database	MySQL 8 (event_ms)
Branch	main

10.2 Project Structure and Package Organization

Folder / File	Description
sql/schema.sql	Tables and sample data
resources/db.properties	Database URL, user and password
resources/web/	Website files
lib/	MySQL Connector/J jar
src/com/eventms/web	WebApp and WebServer
src/com/eventms/controller	Controllers
src/com/eventms/model	Model classes
src/com/eventms/service	Service classes
src/com/eventms/dao	DAO classes
src/com/eventms/exception	Custom exceptions
src/com/eventms/util	DBConnection, ModelFactory, PasswordUtil, Json
javafx/com/eventms	JavaFX Main and UI classes
run-web.bat / run-web.sh	Start the website

10.3 Important Classes and Their Responsibilities

Class	Responsibility
DBConnection	Singleton JDBC connection provider
RegistrationService	Eligibility, clash check, seat check, waiting list and promotion
GamificationService	Points, badges and leaderboard
RecommendationService	Event scoring
BudgetService	Budget and expense validation and approval
FeedbackService	Anonymous feedback and report
Event	Abstract event with clashesWith() and getPointsAwarded()
WebServer	HTTP to controller bridge

10.4 Implementation of Major Modules

10.4.1 Authentication Module

AuthController passes email and password to AuthService, which validates them, hashes the password with PasswordUtil and calls UserDAO. The user's role selects the dashboard.

10.4.2 Event Module

EventService checks title, date, time and capacity; EventDAO saves the event. The event type creates a Workshop, Seminar or Competition object through ModelFactory.

10.4.3 Registration Module

RegistrationService.register() checks eligibility, calls findClash() and either saves the registration, adds a waiting entry or throws ScheduleClashException. cancel() frees the seat and calls promoteFromWaitingList().

10.4.4 Gamification Module

When attendance is marked, GamificationService adds event.getPointsAwarded() to the student and checks badge rules: Newcomer, Active Participant, Event Star and Campus Champion.

10.4.5 Recommendation Module

RecommendationService scores upcoming events and returns a sorted list with reasons.

10.4.6 Budget Module

BudgetService validates amount, purpose and bill, stores the bill under uploads/bills and lets the administrator approve or reject. The summary shows approved budget, expenses and balance.

10.4.7 Feedback Module

FeedbackService verifies attendance and single submission, then FeedbackDAO saves the feedback and the feedback_given record in one transaction. The report shows the average rating and grade.

10.5 Database Configuration and Setup

```
# resources/db.properties
db.url=jdbc:mysql://localhost:3306/event_ms
db.user=root
db.password=your_password
```

10.6 Steps to Run the Application

1. Install JDK 17 or above and MySQL Server.
2. Copy the MySQL Connector/J jar into the lib folder.
3. Run: mysql -u root -p < sql/schema.sql

4. Edit resources/db.properties with the database user and password.
5. Run run-web.bat (Windows) or ./run-web.sh (Linux/macOS).
6. Open http://localhost:8080 in a browser.
7. Log in with a demo account (password pass123): admin@college.edu, organizer@college.edu or asha@college.edu.

11. TESTING

11.1 Testing Methodology

Testing was carried out at three levels: unit testing of service rules (clash, waiting list, points), functional testing through the screens, and integration testing of the full path from the screen to the database. The rules and screens were checked in the browser version with sample data. Tests that need the MySQL server are marked Pending until they are run on the final setup.

11.2 Test Cases

Test ID	Description	Input	Expected Result
TC01	Valid student login	asha@college.edu / pass123	Student dashboard opens
TC02	Invalid login	Wrong password	Error message shown
TC03	Register for eligible event	Asha registers for Java Coding Workshop	Registration saved
TC04	Schedule clash	Asha tries AI Seminar after registering overlapping event	Clash warning, registration blocked
TC05	Full event	Third student registers for capacity-2 event	Student added to waiting list
TC06	Waiting-list promotion	Registered student cancels	First waiting student gets seat and notification
TC07	Attendance and points	Organizer marks attended	Points and badge added
TC08	Budget approval	Admin approves a request	Status Approved, summary updated
TC09	Anonymous feedback	Attendee submits rating 5	Feedback saved without identity
TC10	Duplicate feedback	Same student submits again	Rejected

11.3 Functional Testing

Each module was exercised from the user interface: login and sign-up, event creation, registration and cancellation, attendance, leaderboard, recommendations, budget forms, approvals and feedback. Messages for invalid input were checked for each form.

11.4 Database Testing

Database testing checks that records are inserted, updated and removed correctly, that foreign keys are respected and that the feedback transaction saves both rows or none. It requires a running MySQL server with schema.sql loaded and is to be completed on the final setup.

11.5 Integration Testing

Integration testing follows one request through all layers, for example Register button → RegistrationController → RegistrationService → RegistrationDAO → registrations table. The back-end layers compile together and the web route-to-controller mapping was checked; the complete database path is to be confirmed with MySQL.

11.6 Test Results

Test ID	Actual Result	Status
TC01	Student dashboard opened	Pass (preview)
TC02	Error message shown	Pass (preview)
TC03	Registration shown in My Events	Pass (preview)
TC04	Warning displayed on AI Seminar and registration blocked	Pass (preview)
TC05	Student shown on Waiting List	Pass (preview)
TC06	Seat given and notification "A seat opened up!" shown	Pass (preview)
TC07	Points and Newcomer badge shown on profile	Pass (preview)
TC08	Approved budget Rs.4500, expenses Rs.1800, balance Rs.2700	Pass (preview)
TC09	Report shows average 4.33/5, 3 responses	Pass (preview)
TC10	Rule implemented in FeedbackService	Pending
DB tests	Run on MySQL setup	Pending

Note: "preview" means checked in the browser version that uses in-memory sample data with the same rules.

12. RESULTS & SCREENSHOTS

12.1 Project Results

The project meets all the stated objectives: schedule clashes are detected, cancelled seats move to the waiting list, points and badges are awarded, events are recommended, budgets are approved with bills, and feedback is collected anonymously. The screenshots in this chapter are taken from the browser version of the application with sample data.

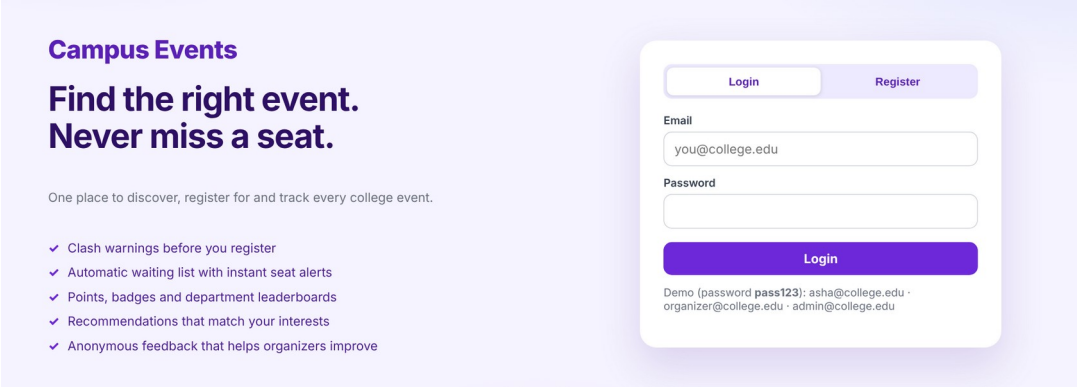


Figure 12.1: Login Page

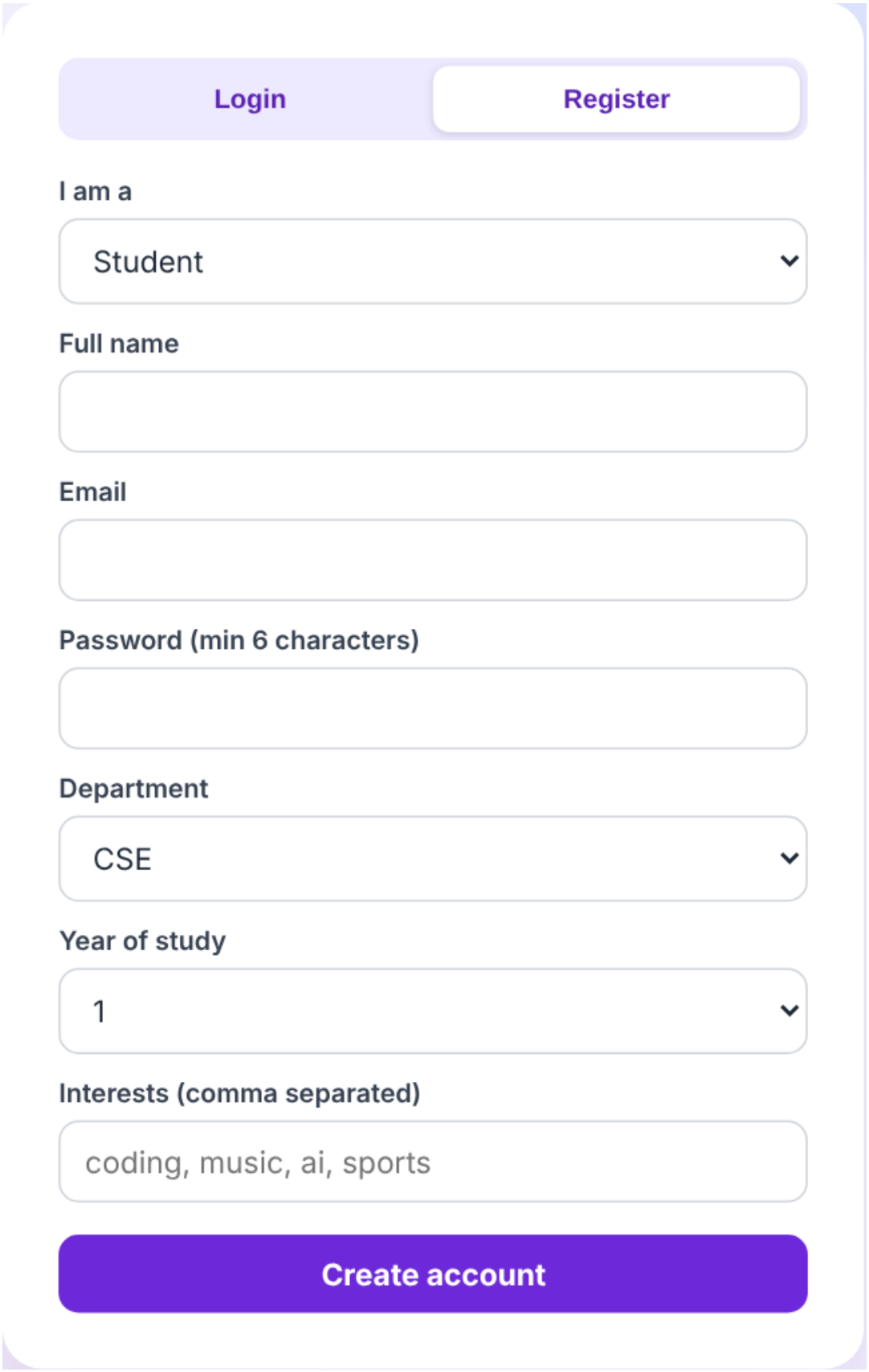


Figure 12.2: Sign-up Page

12.2 Home Page and Dashboard

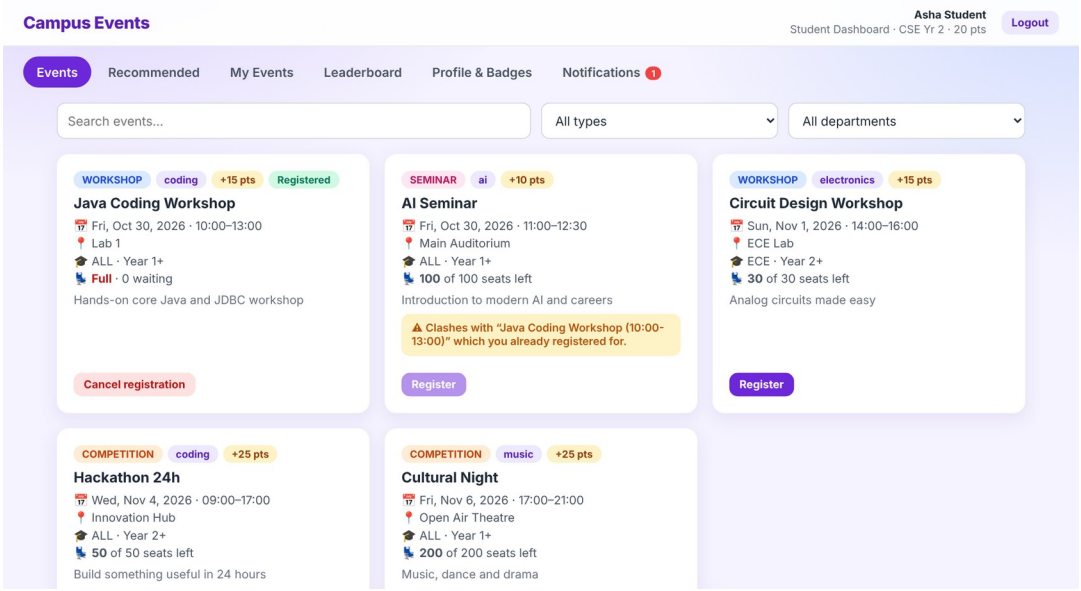


Figure 12.3: Student Dashboard

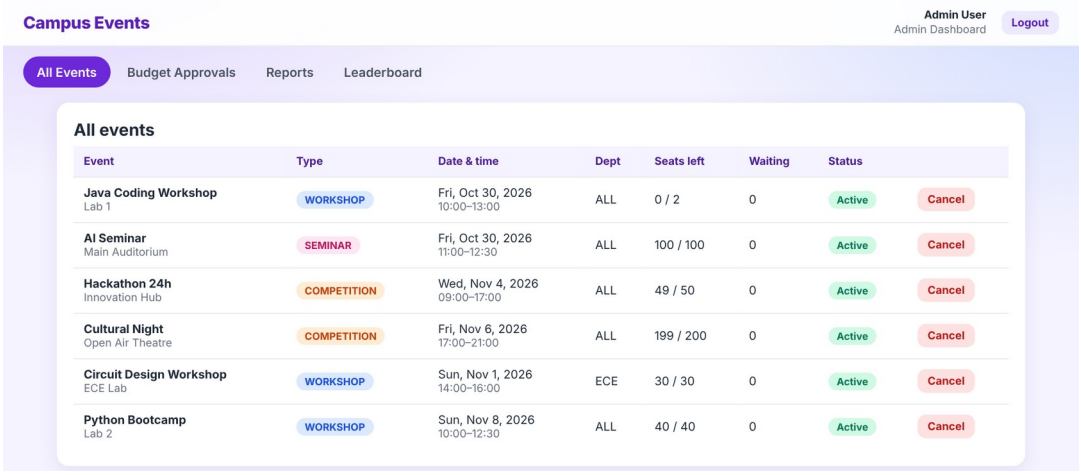


Figure 12.4: Administrator Dashboard

12.3 Event Management Screenshots

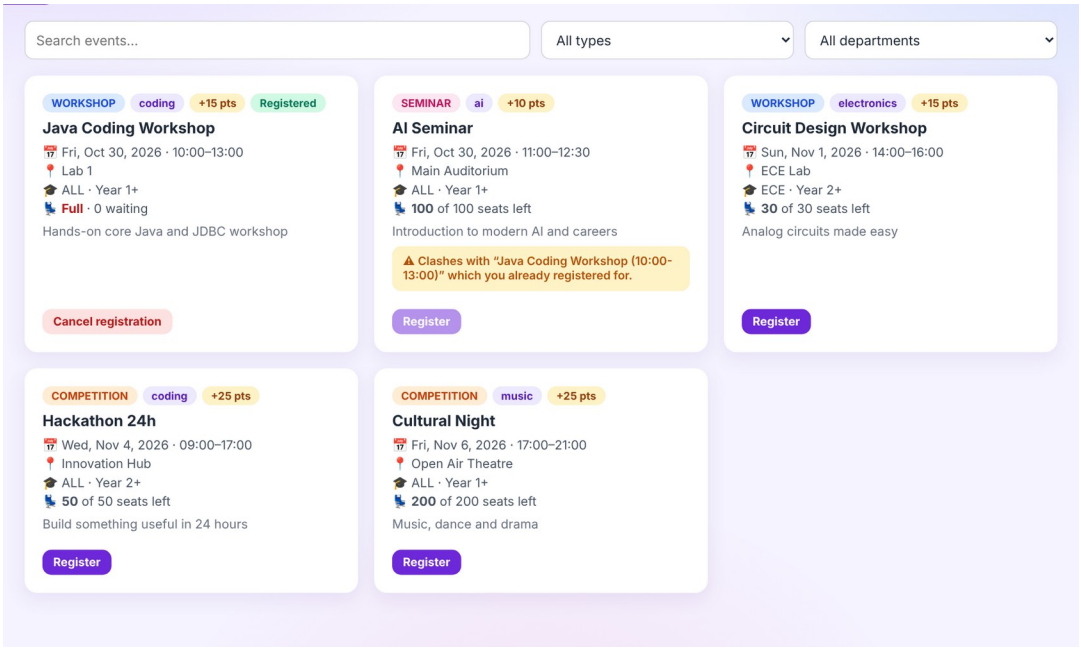


Figure 12.5: Events with Schedule Clash Warning

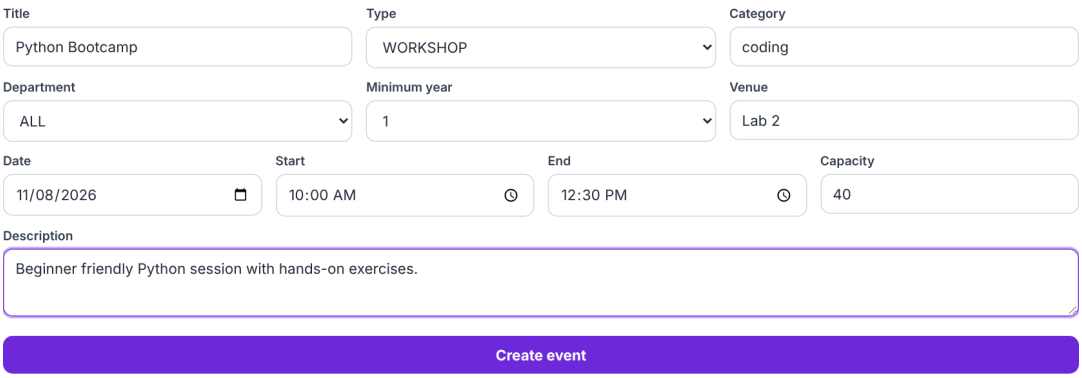


Figure 12.6: Create Event

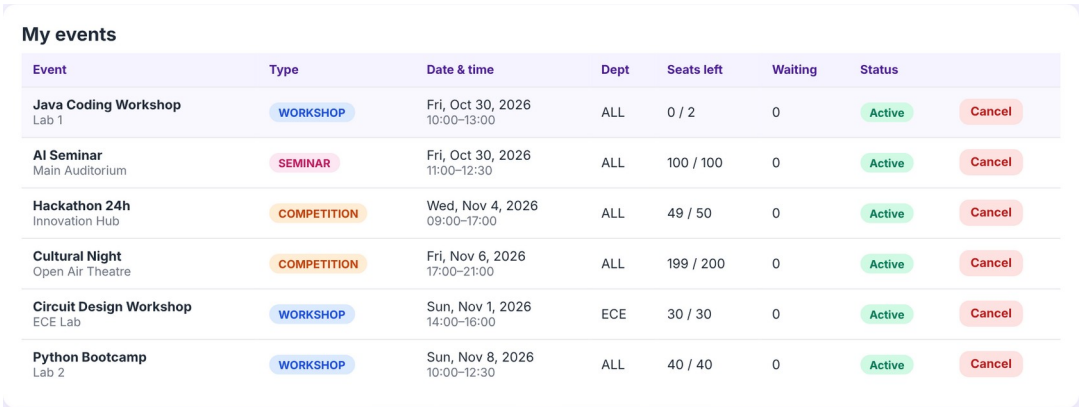


Figure 12.7: Organizer Events

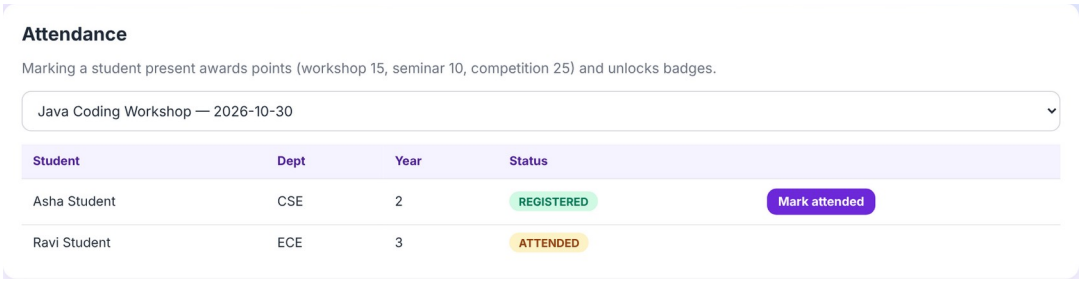


Figure 12.8: Attendance Marking

12.4 Registration Management Screenshots

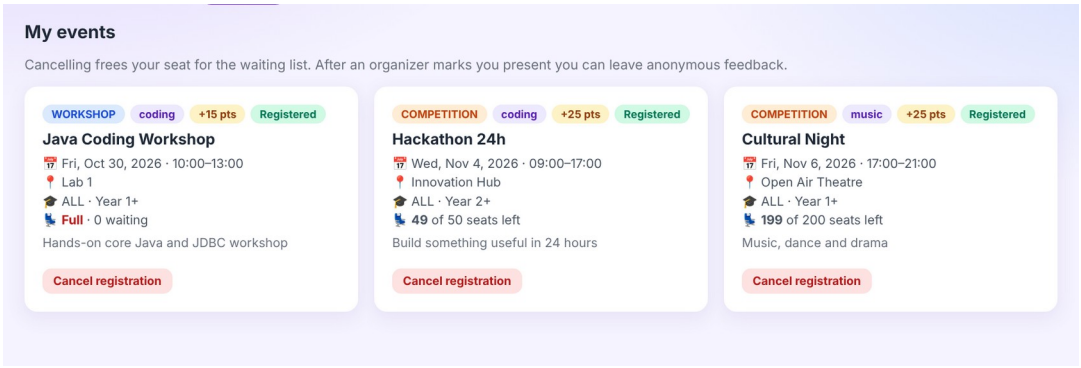


Figure 12.9: My Events

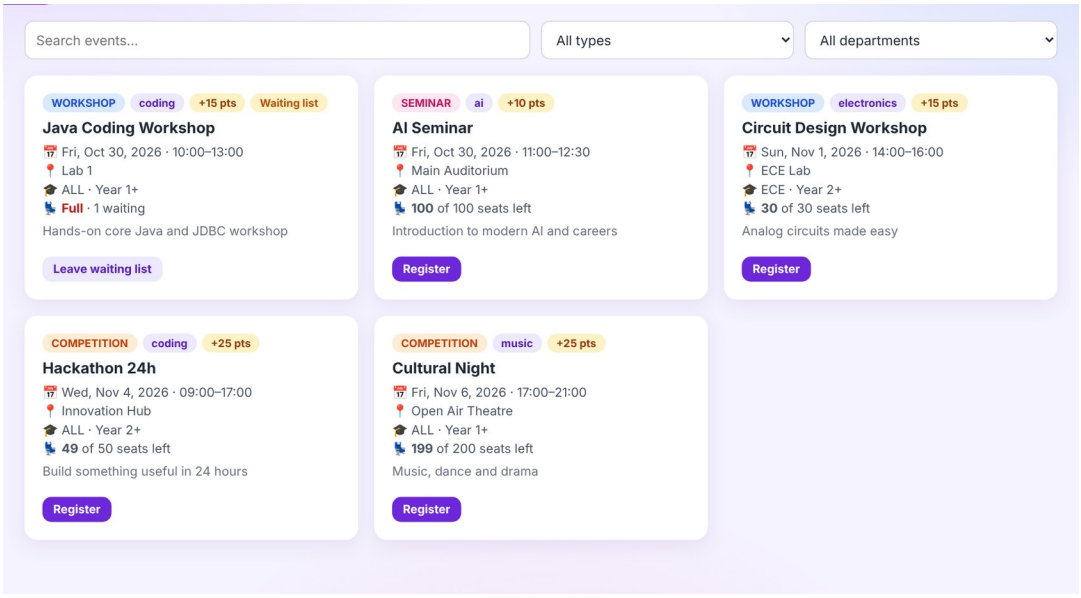


Figure 12.10: Waiting List

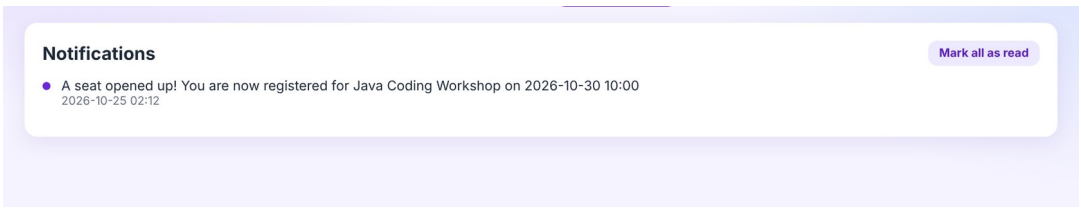


Figure 12.11: Notification after Seat Promotion

12.5 Budget and Feedback Screenshots

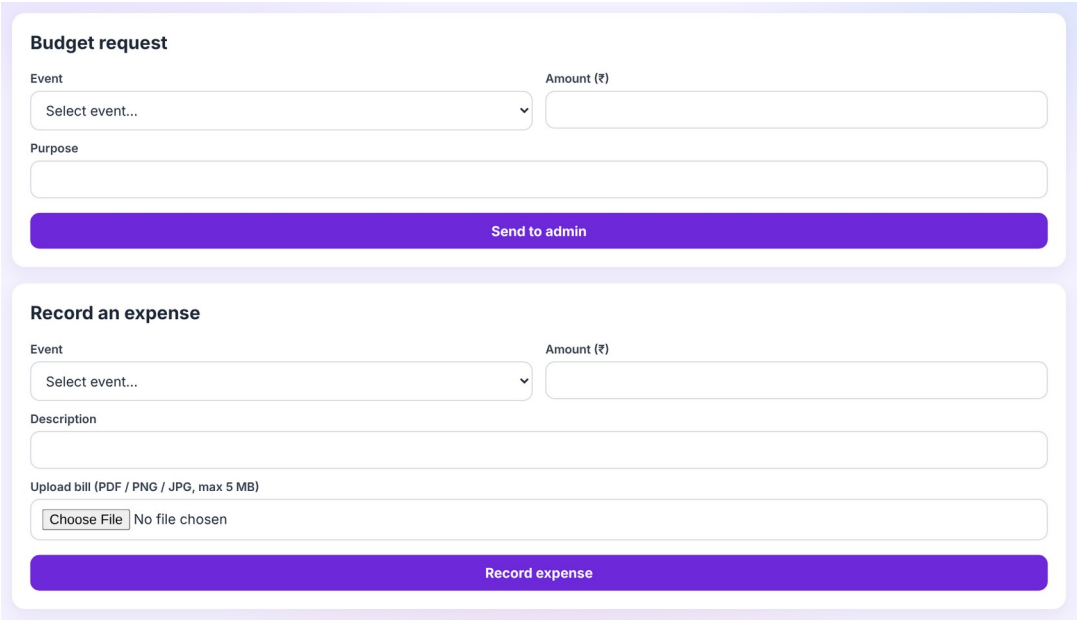


Figure 12.12: Budget Request and Expense Forms

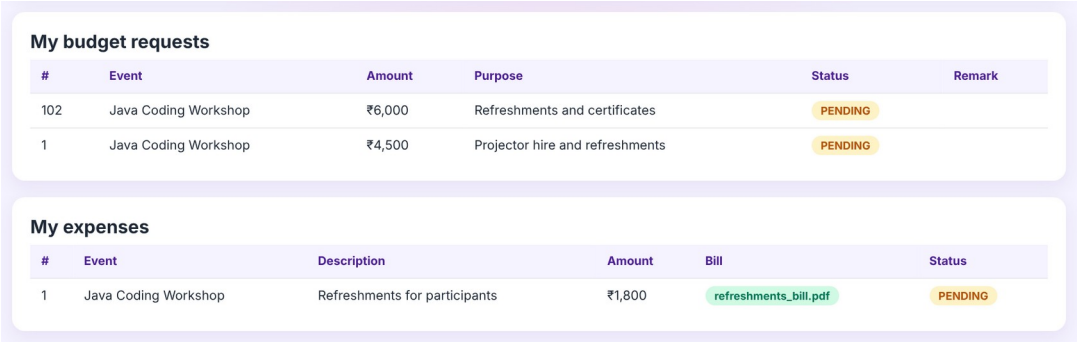


Figure 12.13: Budget and Expense Records

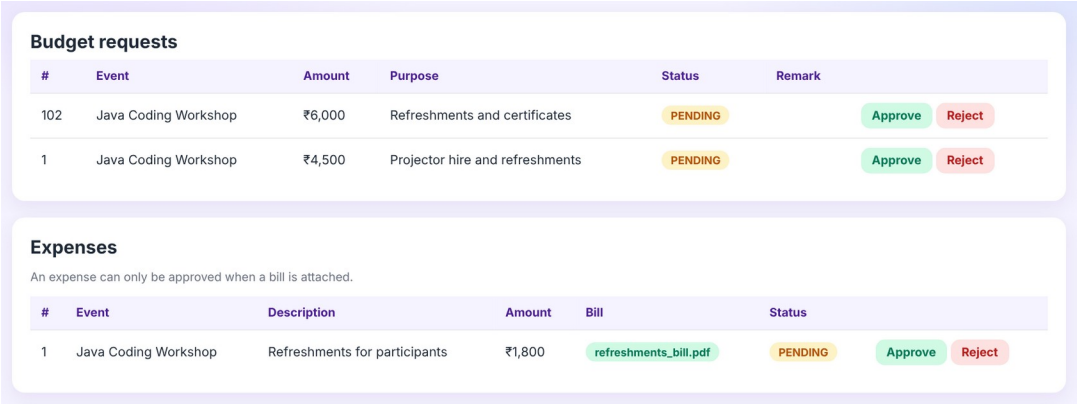


Figure 12.14: Budget Approvals

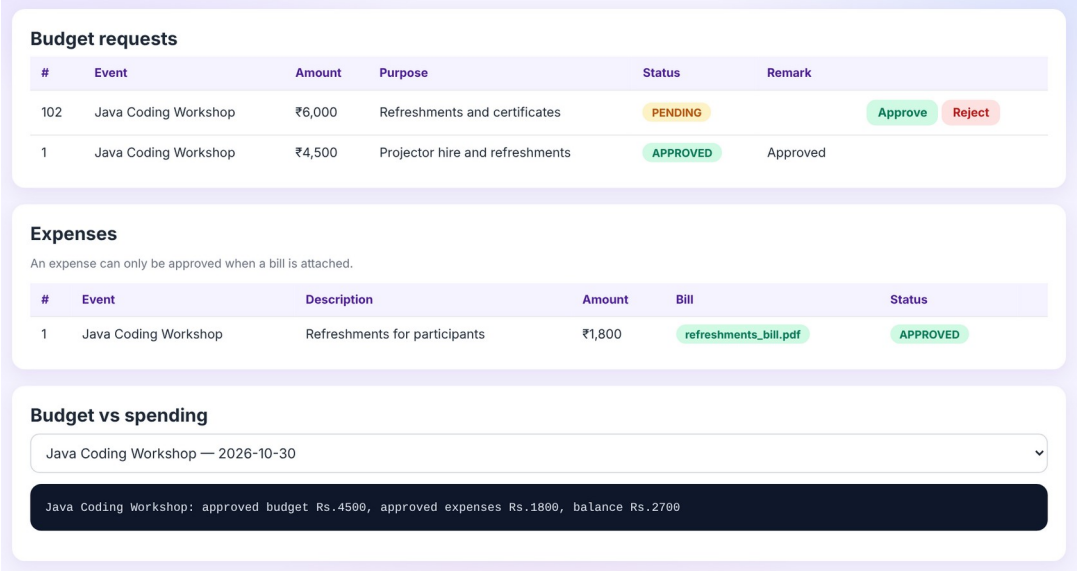


Figure 12.15: Budget versus Spending Summary

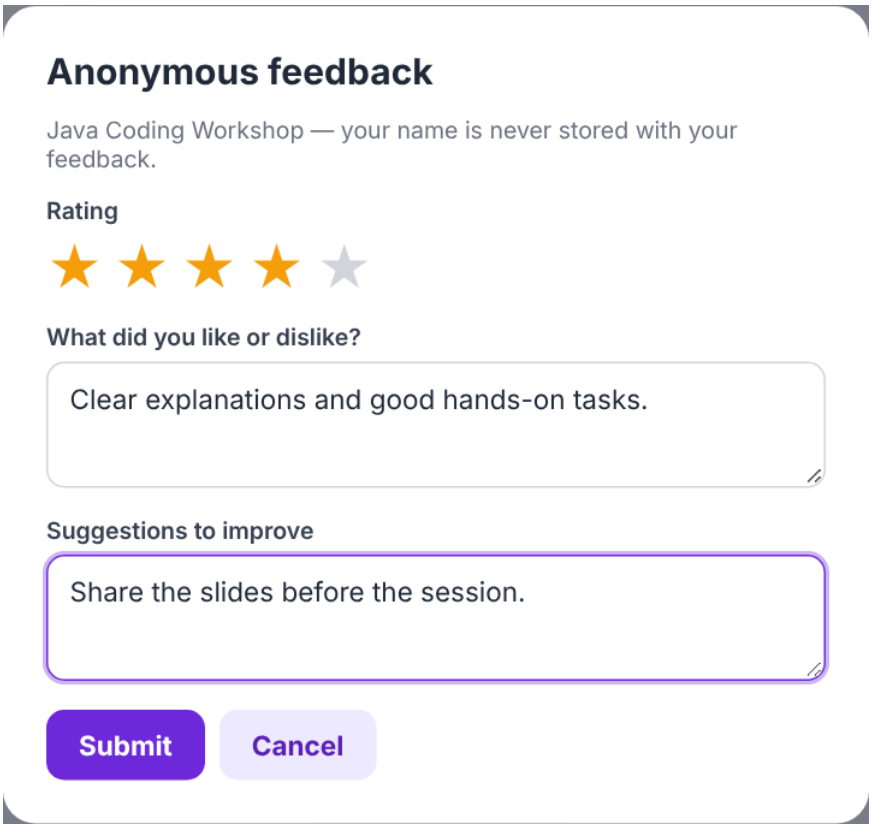


Figure 12.16: Anonymous Feedback Form

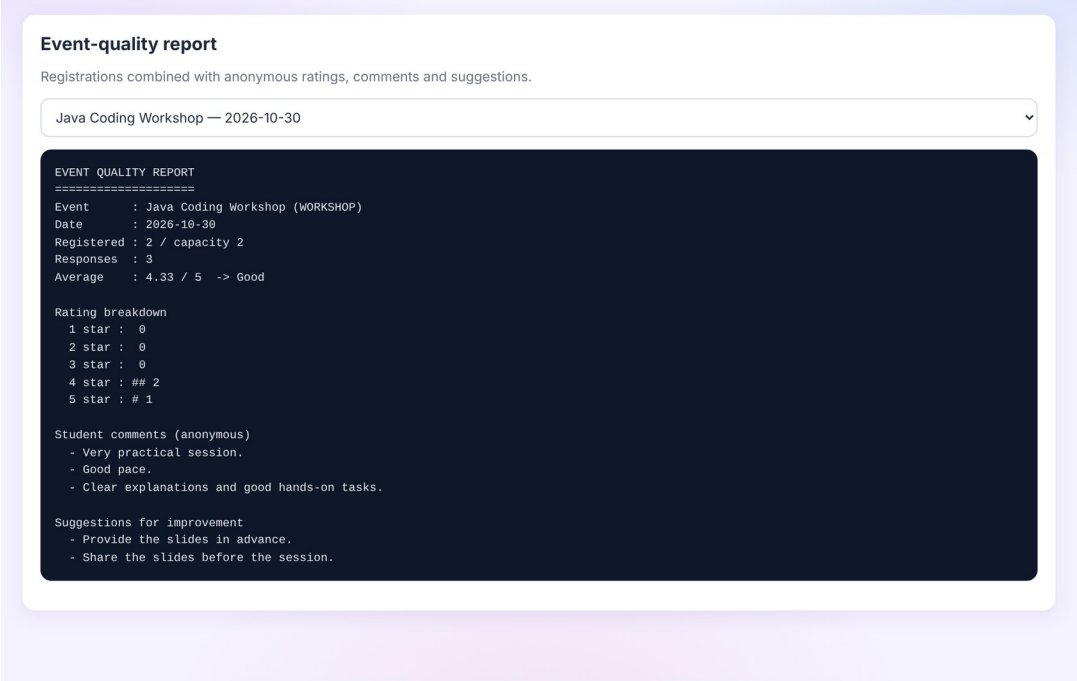


Figure 12.17: Event-Quality Report

12.6 Leaderboard and Recommendation Screenshots

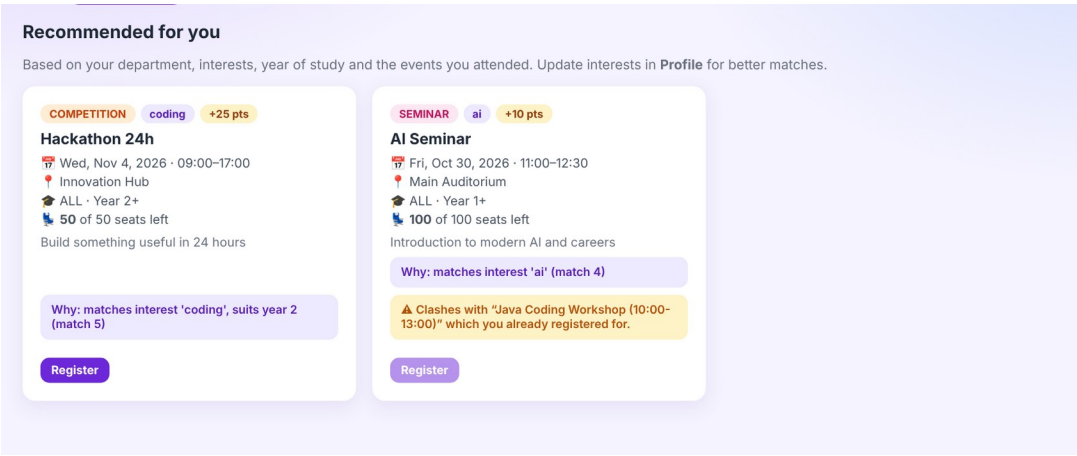


Figure 12.18: Recommended Events

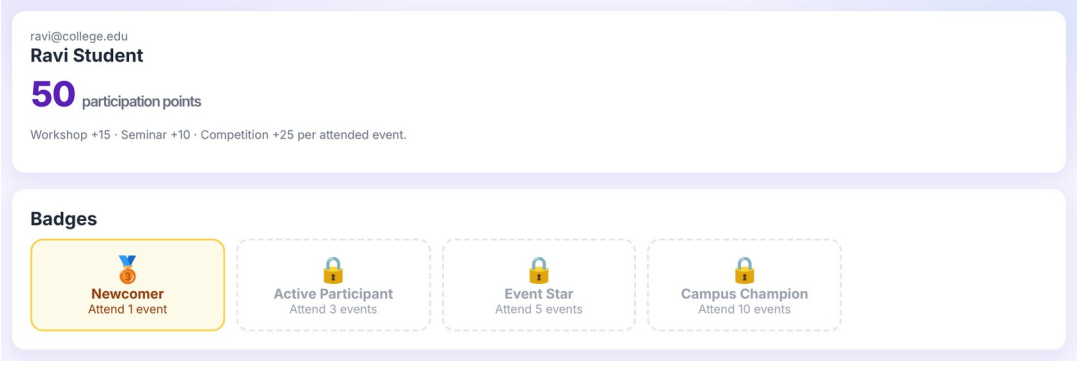


Figure 12.19: Profile with Points and Badge

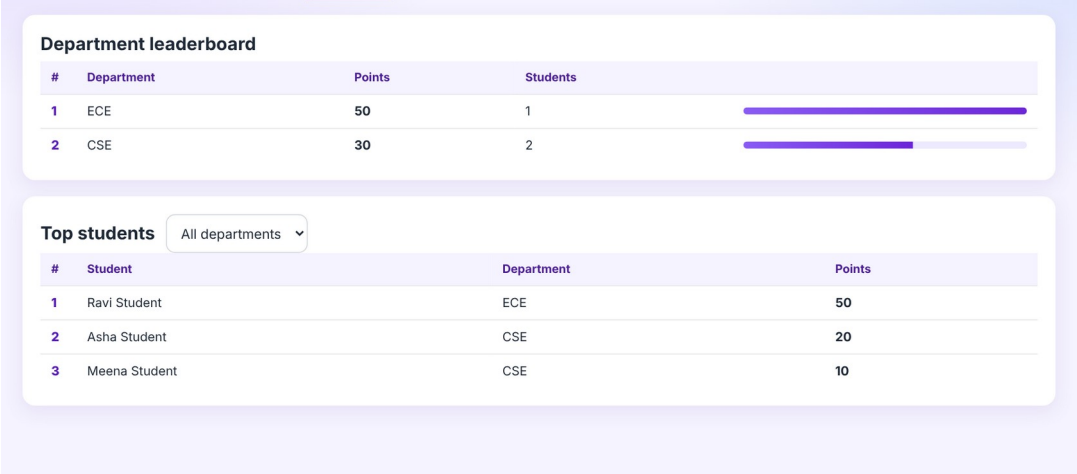


Figure 12.20: Student Leaderboard

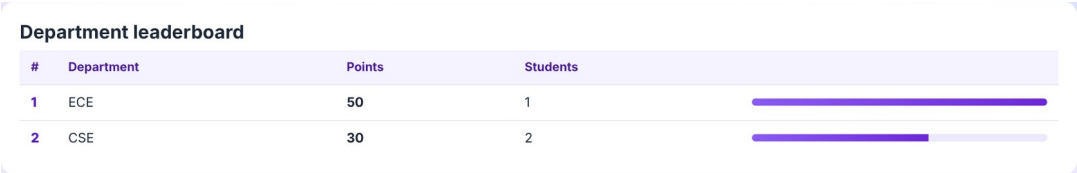


Figure 12.21: Department-wise Leaderboard

13. CONCLUSION

13.1 Project Summary

The Campus Event Management System integrates event creation, registration, waiting list, attendance, rewards, budgeting and feedback in one Java application. It overcomes six drawbacks of a basic event portal by detecting timetable clashes, using cancelled seats through an automatic waiting list, motivating students with points, badges and a leaderboard, recommending events, managing budgets with bill upload and approval, and collecting anonymous feedback with quality reports. The layered design (Controller, Model, Service, DAO) keeps the code organised and allows both a website and a JavaFX interface to share the same back-end.

13.2 Learning Outcomes

1. Applied abstraction, inheritance, polymorphism and encapsulation in a real project.
2. Designed and used a layered architecture.
3. Connected Java to MySQL using JDBC and PreparedStatement.
4. Designed a relational database with primary and foreign keys.
5. Used transactions to keep related records consistent.
6. Created custom exception classes and validated input.
7. Used the Collections framework and Comparator for sorting.
8. Applied Singleton and Factory design patterns.
9. Built a browser interface and a JavaFX interface over one back-end.
10. Planned, tested and documented a complete software project.

14. FUTURE ENHANCEMENT

14.1 AI-Based Personalized Event Recommendations

The present rule-based scoring can be replaced or supported by a machine-learning model trained on past registrations, attendance and ratings, so that suggestions improve as more data is collected. Sentiment analysis of feedback comments can also be added to the event-quality report.

14.2 Automated Email and SMS Notifications

Notifications are at present shown inside the application. Future versions can send e-mail and SMS messages for seat promotion from the waiting list, budget approval, event reminders and certificate availability.

14.3 Smart Student Preference and Personalized Event Assistant

A smart assistant can learn each student's preferences, suggest events that fit their free time by linking the class timetable, and answer questions about events. Other possible additions are QR-code attendance, online payment for paid events, automatic e-certificates, a mobile application, REST APIs with a framework such as Spring Boot, two-factor authentication and deployment on a college server with HTTPS.